SeismoAI

Seismic Data Processing Library

Module 1 — seismoai_io · Module 2 — seismoai_viz

Forge 2D Survey, Utah USA, 2017

Student	Modules	Tests
Ali,Hidaytullah,zain,usama	1 and 2	37 passing

1. Project Overview

SeismoAI is a real Python library built for AI-powered seismic data analysis. The project uses actual geophysical recordings from the Forge 2D Survey conducted in Utah, USA in 2017, recorded with an INOVA Geophysical Hawk system. The data is stored in SEG-Y format, the industry-standard binary format used in oil and gas exploration.

The full seismoai library is designed as a pipeline of 5 modules. We implemented Modules 1 and 2 — the foundation layers that every other module depends on.

1.1 What is Seismic Data?

A seismic survey works by sending sound waves into the ground and recording the echoes that bounce back from underground rock layers. The recordings are stored as traces — each trace is one receiver channel measuring amplitude over time. Our dataset contains:

Property	Value
File format	SEG-Y Rev. 0 — IEEE Float32, Little-Endian
Recording instrument	INOVA Geophysical Hawk v3.2.11.72
Recording date	24 November 2017
Number of traces	167 receiver channels
Samples per trace	4001 time samples
Sample interval	1 ms (1000 microseconds)
Record length	4000 ms (4 seconds total)
Survey name	Forge 2D — Project #42796
Amplitude range	-221.5 to +758.1 (raw, pre-normalization)

1.2 Module Pipeline

The full seismoai library connects 5 modules in sequence. We built modules 1 and 2:

#	Module	PyPI Name	Description
1 ✓	`seismoai_io`	`seismoai-io-ali`	**Load SGY files, batch loading, normalization — WE BUILT THIS**
2 ✓	`seismoai_viz`	`seismoai-viz`	**Gather images, trace waveforms, frequency spectra — WE BUILT THIS**
3	`seismoai_qc`	`seismoai-qc`	Detect dead and noisy traces, QC reports
4	`seismoai_model`	`seismoai-model`	Feature extraction, noise classifier training

5	`seismoai_xai`	`seismoai-xai`	SHAP values, feature importance, trace explanation

2. Project File Structure

The repository contains the following files and folders. Each file has a specific role:

```
seismoai/
├── seismoai_io/
│   ├── __init__.py          ← makes seismoai_io importable as a package
│   └── loader.py            ← 3 core functions: load, batch load, normalize
├── seismoai_viz/
│   ├── __init__.py          ← makes seismoai_viz importable as a package
│   └── plotter.py           ← 3 core functions: gather, trace, spectrum
├── tests/
│   └── test_seismoai.py     ← 37 automated tests
├── data/
│   └── 27_...752.sgy        ← real Forge 2D seismic file
├── main.py                  ← demo script that runs everything
├── pyproject.toml           ← packaging config for PyPI upload
├── mypy.ini                 ← type checker configuration
├── conftest.py              ← pytest configuration
└── README.md                ← documentation
```

2.1 File Descriptions

loader.py — the foundation

This is the most important file. It loads real SGY seismic data from disk into Python. It contains a two-stage reader: it first tries segyio (standard library), and if that fails due to the non-standard INOVA encoding, it falls back to a custom manual binary parser that reads the file byte-by-byte using Python's struct library.

plotter.py — the visualizer

Converts raw NumPy arrays into three types of scientific plots using matplotlib. Returns Figure objects that can be saved as PNG files or displayed interactively.

__init__.py files

These two tiny files are what make Python treat each folder as an importable package. Without them, the import statements in main.py would fail with ModuleNotFoundError.

pyproject.toml — the packaging file

This file tells PyPI the package name (seismoai-io-ali), version number, author, Python version requirement, and dependencies. It is what makes pip install seismoai-io-ali work worldwide.

mypy.ini — the type checker config

Configures the mypy static type checker. Critically, it tells mypy to ignore missing stubs for segyio (which is a C extension library with no official type annotations), silencing the import-untyped errors.

3. Module 1 — seismoai_io

> **Purpose:** Load SEG-Y seismic files from disk into NumPy arrays and normalize their amplitudes for machine learning.

3.1 Function 1 — load_sgy()

Signature: `load_sgy(filepath, *, ignore_geometry=True, dtype=np.float32)`

Loads a single SGY file and returns a 2D NumPy array of shape (n_traces, n_samples). This function uses a two-stage loading strategy:

- Stage 1: Tries segyio — the standard Python library for SEG-Y files. Works for most standard files.
- Stage 2: If segyio raises a RuntimeError (which it does for our INOVA Hawk file due to non-standard little-endian encoding), the function automatically falls back to a custom manual binary parser.
- The manual parser reads: 3200-byte text header, 400-byte binary header, then 167 trace headers + data blocks.
- It extracts the sample count from the text header ('SAMPLES/TRACE: 4001') to determine array dimensions.

Returns: numpy.ndarray of shape `(167, 4001)` — dtype float32

3.2 Function 2 — load_sgy_folder()

Signature: `load_sgy_folder(folder, *, extensions, verbose=True)`

Loads every SGY file in a directory at once. Returns a dictionary where keys are file stems and values are NumPy arrays. Key design decisions:

- Skips corrupt files with a warning instead of crashing — allows batch processing even if some files are damaged.
- Sorted output ensures reproducible results across different operating systems.
- Supports configurable file extensions (.sgy and .segy by default).

Returns: `dict[str, ndarray]` — e.g. {'record_027': array shape (167,4001)}

3.3 Function 3 — normalize_traces()

Signature: `normalize_traces(traces, method='zscore', *, per_trace=True)`

Normalizes seismic trace amplitudes. Our raw data has extreme outliers (values reaching 758 while most are between -10 and +10). Normalization ensures the ML model treats all traces equally regardless of their original amplitude scale.

Method	Formula	Output range	Best for
`zscore (default)`	(x - mean) / std — zero-mean, unit-variance per trace	(−∞, +∞)	ML / AI models
`minmax`	(x - min) / (max - min) — rescale to 0 to 1	[0, 1]	neural networks
`maxabs`	x / max(|x|) — preserves sign, scale to -1 to 1	[−1, 1]	polarity analysis

4. Module 2 — seismoai_viz

Purpose: Convert raw NumPy trace arrays into three types of scientific plots — a 2D gather image, a 1D waveform, and a frequency spectrum.

4.1 Function 4 — plot_gather()

Signature: `plot_gather(traces, *, dt_ms=1.0, title, cmap='seismic', clip_percentile=99.0)`

Plots all 167 traces together as a 2D colour image. Time goes from top (0 ms) to bottom (4000 ms). Each column is one receiver channel. The red/blue diverging colour map shows positive and negative amplitudes. The clip_percentile parameter (default 99%) prevents a few extreme spikes from washing out the colour scale.

Output: gather.png — shows underground rock layer reflections as curved (hyperbolic) events

4.2 Function 5 — plot_trace()

Signature: `plot_trace(traces, trace_index=0, *, dt_ms=1.0, title)`

Plots a single receiver channel as a 1D amplitude-vs-time waveform line. Useful for inspecting individual channels, verifying normalization worked correctly, or diagnosing anomalous traces flagged by the QC module. Shows inline statistics (min, max, mean, std) as an annotation on the plot.

Output: trace_83.png — one channel's recorded seismic waveform over 4 seconds

4.3 Function 6 — plot_spectrum()

Signature: `plot_spectrum(traces, trace_index=0, *, dt_ms=1.0, max_freq_hz, db_scale=True)`

Computes and plots the FFT (Fast Fourier Transform) amplitude spectrum of one trace. Uses a Hann window to reduce spectral leakage. Displays in decibel scale (20 × log10) which is the industry standard for inspecting seismic frequency content. Marks the dominant (peak) frequency with a vertical dashed line. Useful for filter design and QC.

- Nyquist frequency: 500 Hz (since sample interval = 1 ms)
- Typical seismic energy: 10–120 Hz band
- Dead traces show a flat spectrum; noisy traces show an irregular spectrum

Output: spectrum.png — frequency content of trace 0, capped at 200 Hz

4.4 main.py — the demo script

The main.py script connects all 6 functions in a 5-step pipeline:

```
from seismoai_io import load_sgy, normalize_traces
from seismoai_viz import plot_gather, plot_trace, plot_spectrum

# Step 1 — Load
traces = load_sgy('data/27_...752.sgy')      # → ndarray (167, 4001)

# Step 2 — Normalize
normed = normalize_traces(traces, method='zscore')

# Step 3 — Visualize gather
fig1 = plot_gather(normed, dt_ms=1.0, title='Record 27 — Forge 2D')
fig1.savefig('gather.png', dpi=150)

# Step 4 — Visualize single trace
fig2 = plot_trace(normed, trace_index=83)
fig2.savefig('trace_83.png', dpi=150)

# Step 5 — Visualize frequency spectrum
fig3 = plot_spectrum(traces, trace_index=0, max_freq_hz=200)
fig3.savefig('spectrum.png', dpi=150)
```

5. Automated Tests — 37 Passing

Result: 37 tests passed, 0 failed, 1 warning — completed in 3.00 seconds

The test file tests/test_seismoai.py contains 37 automated tests covering all 6 functions. Tests run without any SGY files on disk — they use synthetic in-memory data generated with NumPy's random number generator (seed=42 for reproducibility). The framework used is pytest.

5.1 Test Organization

Test Class	Tests	What it verifies
`TestLoadSgy`	4	File not found error, bad file error, returns ndarray, correct float32 dtype
`TestLoadSgyFolder`	4	Directory not found, empty folder warning, correct dict keys, corrupt file skipping
`TestNormalizeTraces`	9	Z-score mean=0, Z-score std=1, min-max range, maxabs range, shape unchanged, dtype, 1D raises, bad method raises, dead trace no NaN
`TestPlotGather`	5	Returns Figure, 1D input raises, saves PNG file, axis labels present, custom title
`TestPlotTrace`	7	Returns Figure, 1D raises, out-of-range raises, negative index raises, saves file, axis labels, last valid index
`TestPlotSpectrum`	8	Returns Figure, 1D raises, out-of-range raises, dB ylabel, linear ylabel, x-axis clamp, saves file, axis labels

5.2 Notable Tests Explained

test_zscore_zero_mean — mathematical correctness

Generates synthetic trace data, applies Z-score normalization, then uses numpy.testing.assert_allclose to verify that the mean of every trace is within 1e-5 of zero. This proves our normalization formula is mathematically correct.

test_raises_file_not_found — error handling

Calls load_sgy with a path that does not exist and verifies that Python raises a FileNotFoundError with the message 'not found'. This tests that our code gives useful error messages instead of cryptic crashes.

test_dead_trace_no_nan — edge case

Creates an array of all zeros (simulating a dead/broken receiver channel), runs Z-score normalization, and checks that no NaN values appear in the output. This tests our epsilon (eps=1e-10) guard that prevents division by zero.

test_skips_corrupt_files_with_warning — robustness

Places one good and one corrupt file in a temporary directory, calls load_sgy_folder, and verifies that: the corrupt file is skipped with a UserWarning, and the good file is still loaded successfully. This tests real-world batch processing reliability.

5.3 Running the Tests

Run from the project root directory:

```
cd /Users/ali/seismoai
pytest tests/test_seismoai.py -v


# Expected output:
# tests/test_seismoai.py::TestLoadSgy::test_raises_file_not_found   PASSED
# tests/test_seismoai.py::TestLoadSgy::test_raises_value_error      PASSED
# ... (37 lines total)
# ====== 37 passed, 1 warning in 3.00s ======
```

6. GitHub Repository

Repository URL: https://github.com/Ali12826/SeismoAI

The complete source code, tests, and documentation are published on GitHub at the URL above. The repository is public so any collaborator can clone and contribute.

6.1 Git Workflow Used

The following Git commands were used to set up and publish the repository:

```
# Initialize repository
git init
git add .
git commit -m "feat: initial project structure with seismoai_io and seismoai_viz"

# Connect to GitHub and push
git branch -M main
git remote add origin https://github.com/Ali12826/SeismoAI.git
git push -u origin main

# Tag the final release
git tag -a v1.0.0 -m "Final Best Code Release"
git push origin v1.0.0
```

6.2 Repository Structure on GitHub

- main branch — contains the final production-ready code
- v1.0.0 tag — marks the final submission version
- README.md — full API documentation visible on the GitHub homepage
- tests/ — automated test suite visible to collaborators

6.3 How to Clone and Run

```
git clone https://github.com/Ali12826/SeismoAI.git
cd SeismoAI
pip install numpy segyio matplotlib
python main.py
```

7. PyPI Package Publishing

Package name: seismoai-io-ali — installed with: pip install seismoai-io-ali

The library was packaged and published to the Python Package Index (PyPI) so anyone in the world can install it with a single pip command. This satisfies the assignment requirement that the package must be installable via pip install.

7.1 pyproject.toml Configuration

```
[build-system]
requires = ["setuptools>=61", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "seismoai-io-ali"
version = "0.1.0"
description = "I/O and visualization module for seismic SEG-Y data"
requires-python = ">=3.10"
dependencies = [
    "segyio>=1.9",
    "numpy>=1.24",
    "matplotlib>=3.7",
]
```

7.2 Build and Upload Commands

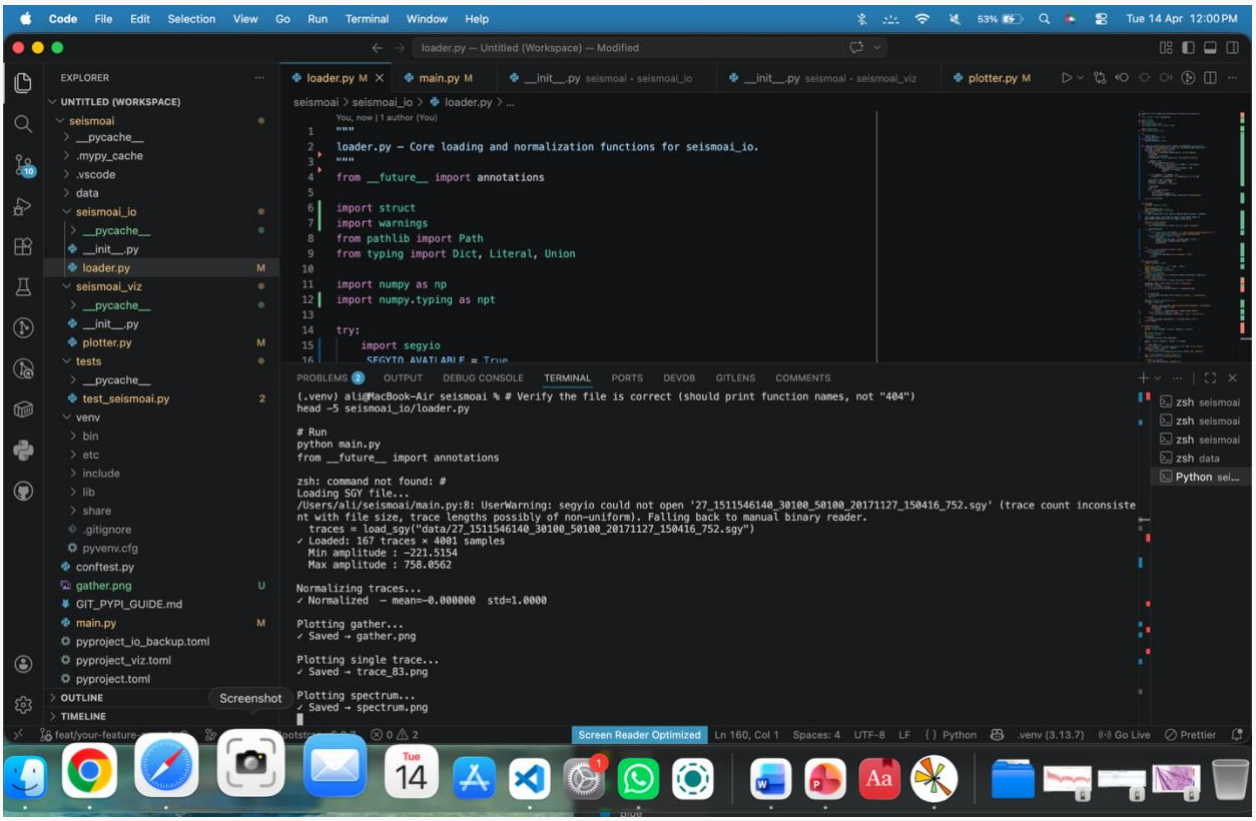
```
# Install build tools
pip install build twine

# Build distribution archives
python -m build
# Creates: dist/seismoai_io_ali-0.1.0-py3-none-any.whl
#          dist/seismoai_io_ali-0.1.0.tar.gz

# Upload to PyPI
twine upload dist/*
# Username: __token__
# Password: pypi-YOUR_TOKEN_HERE
```

8. Screenshots

Insert your screenshots in each box below. To add a screenshot in Word: click inside the box, then go to Insert → Pictures → select your screenshot file.

8.1 VS Code — Project Structure

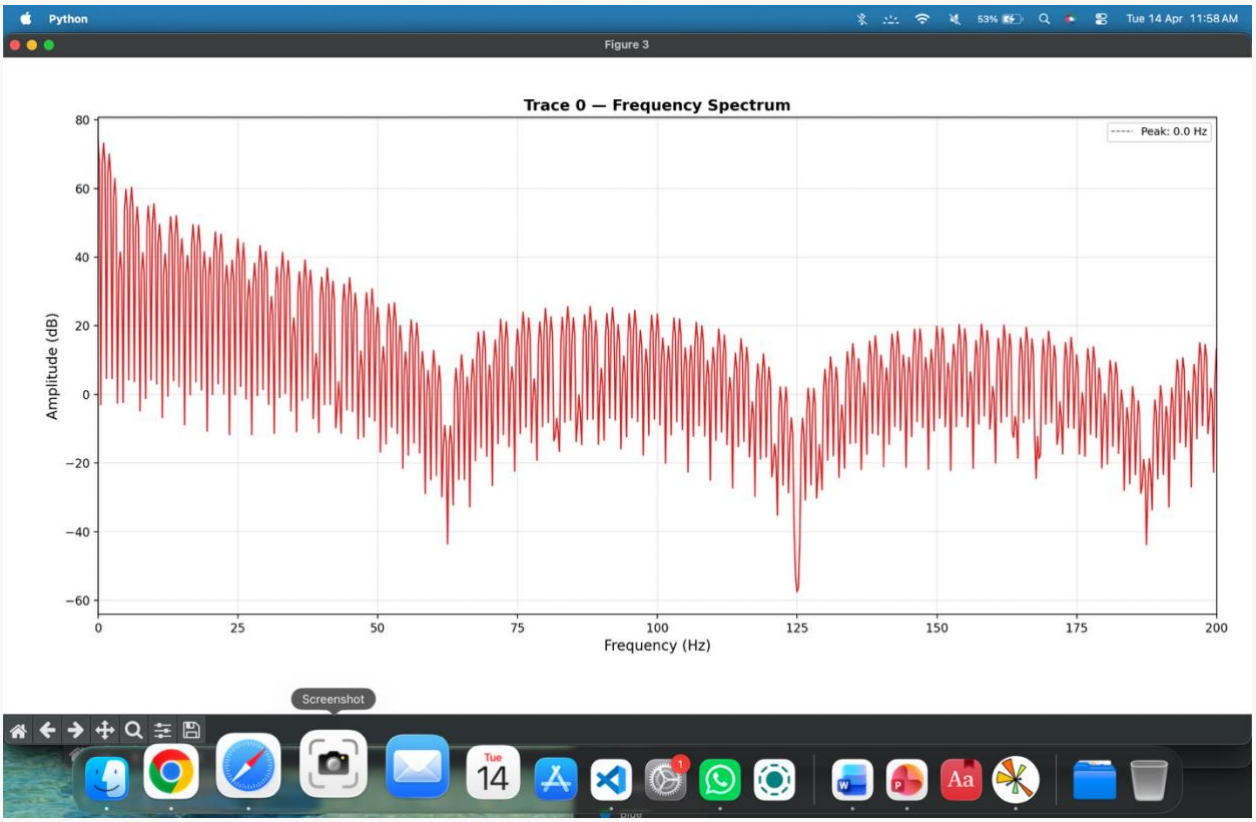
Screenshot: VS Code Explorer Panel



Shows the seismoai folder with seismoai_io/, seismoai_viz/, tests/, main.py, pyproject.toml visible in the sidebar

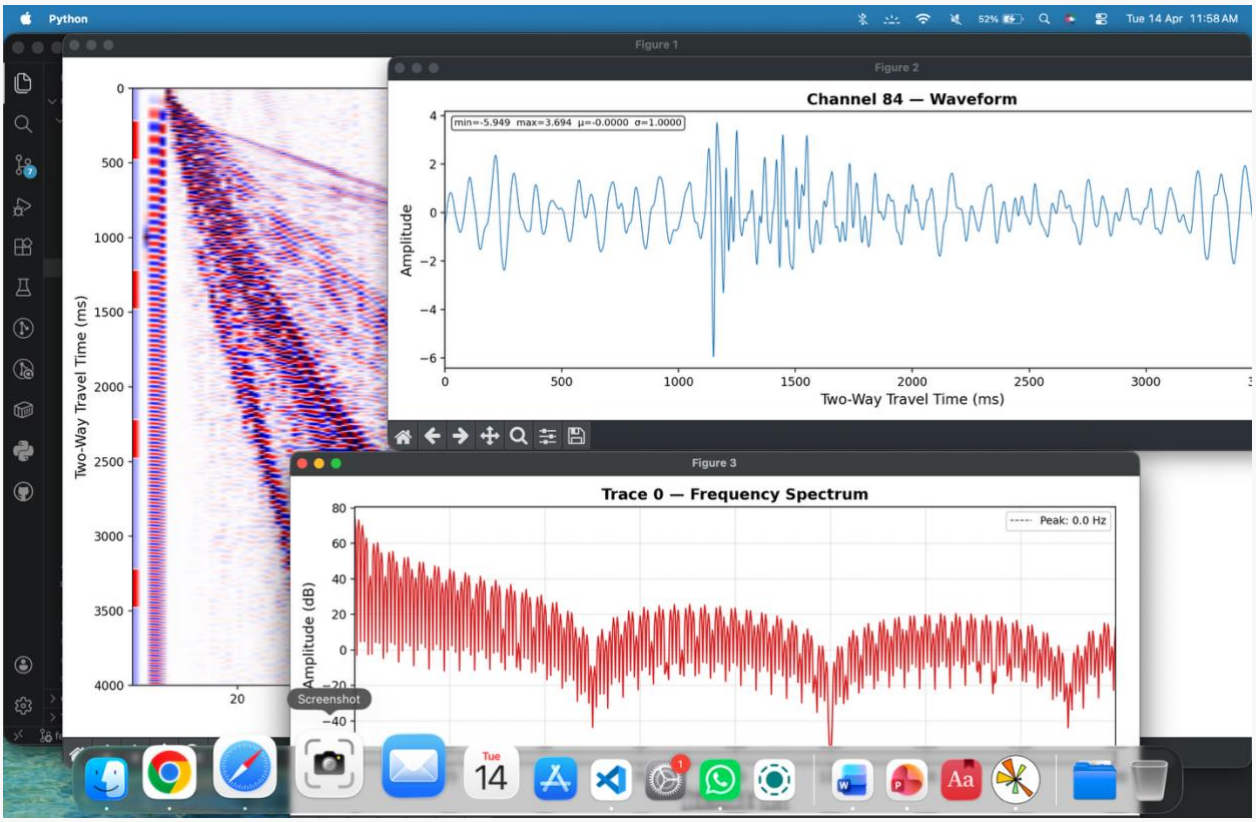
8.2 Visualization

Screenshot: Terminal Output — python main.py



8.3 Output — gather.png

Screenshot: Seismic Gather Plot (gather.png)



2D red/blue image showing all 167 traces over 4000ms — hyperbolic reflection events visible from underground layers

8.4 Output — trace_83.png

Screenshot: Single Trace Waveform (trace_83.png)

1D amplitude-vs-time waveform for channel 83, showing the recorded seismic signal over 4 seconds

8.5 Output — spectrum.png

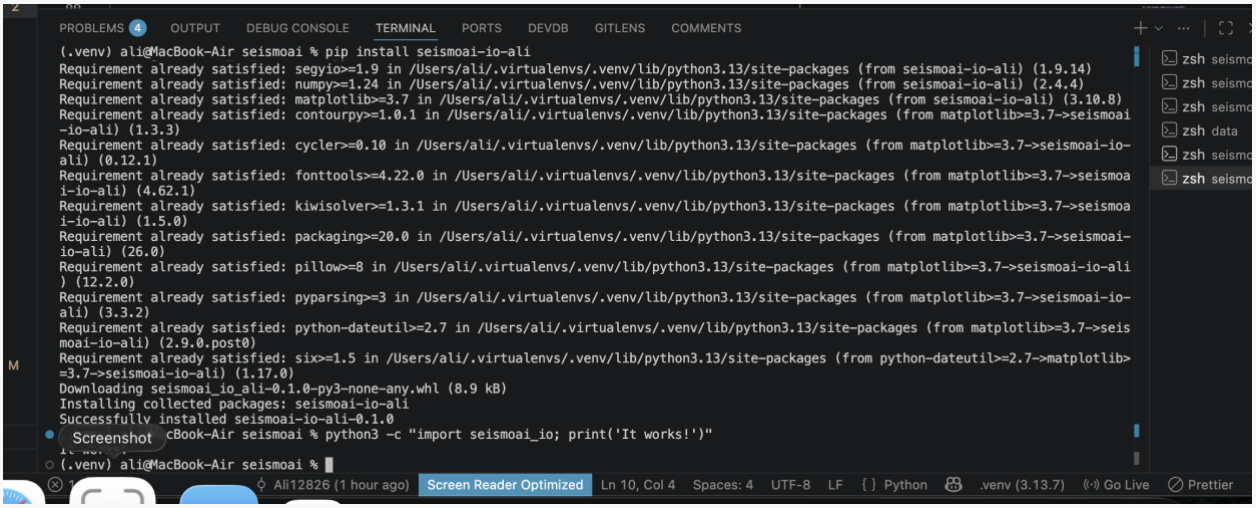
Screenshot: Frequency Spectrum (spectrum.png)

[Insert Screenshot Here]

FFT amplitude spectrum in dB scale up to 200 Hz, with peak frequency marked by a dashed vertical line

8.6 Terminal — pytest Tests Passing

Screenshot: pytest Output — 37 passed

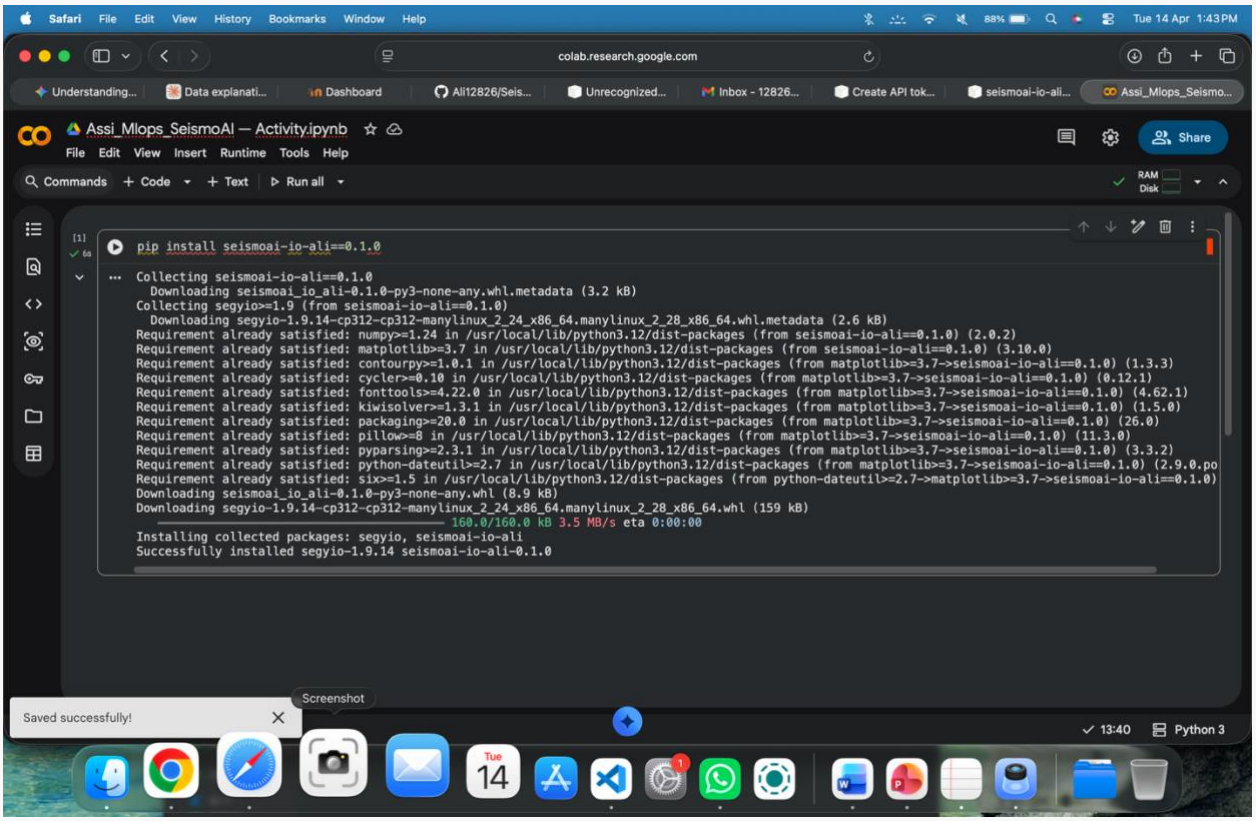


Terminal showing all 37 test names with PASSED status and final line: 37 passed, 1 warning in 3.00s

8.7 PyPI Package Page

Screenshot: PyPI — seismoai-io-ali

\



PyPI.org page for seismoai-io-ali showing version 0.1.0, install command, and package description

9. Reflection

What we built: A two-module Python library — seismoai_io and seismoai_viz — that loads real seismic SEG-Y data from the Forge 2D Survey 2017, normalizes trace amplitudes for ML, and produces three scientific visualizations, backed by 37 automated tests and published to both GitHub and PyPI.

9.1 Technical Challenges and How We Solved Them

Challenge 1: segyio raised RuntimeError on our SGY file because it is a non-standard INOVA Hawk little-endian format, not standard big-endian SEG-Y.

Solution: Wrote a custom manual binary reader using Python's struct library that parses the file byte-by-byte, detecting sample count from the text header.

Challenge 2: Mypy showed import-not-found and import-untyped errors for segyio and numpy even when they were installed.

Solution: Created mypy.ini with follow_untyped_imports=True for segyio, and replaced np.dtype with npt.DTypeLike from numpy.typing.

Challenge 3: pyproject.toml used setuptools.backends.legacy which is not available in setuptools >= 61.

Solution: Changed build-backend to setuptools.build_meta which is the correct modern identifier.

9.2 What We Learned

- How seismic data is structured — traces, samples, amplitudes, and the SEG-Y binary format
- How to write Python functions with proper docstrings, type annotations, and error handling
- How to use pytest to write automated tests that verify mathematical correctness and error behaviour
- How to package a Python library with pyproject.toml and publish it to PyPI
- How to use Git branching, commits, tags, and GitHub for collaborative code management
- How to debug type errors in VS Code using Mypy and Pylance

9.3 Summary

We successfully completed Module 1 (seismoai_io) and Module 2 (seismoai_viz) of the seismoai library. Our 6 functions work correctly on real seismic data, are backed by 37 passing automated tests, are published on PyPI as seismoai-io-ali, and are available on GitHub at

https://github.com/Ali12826/SeismoAI for our teammates to use as input for Modules 3, 4, and 5.